



윤경은

GenON AI Engineer Intern

yge0307@gmail.com
github.com/ykgstar37-lab
yge-portfolio.vercel.app

LLM Agent 신뢰성과 RAG 품질을 데이터로 검증해 고객 솔루션에 연결하는 AI 엔지니어

ABOUT

지원동기

GenON이 GenOS 플랫폼과 고객사 커스터마이징을 통해 sLLM 솔루션을 실서비스에 안착시키는 방향성에 끌렸습니다. WorkFlow Agent에서 sLLM 과신 응답을 검증 게이트로 차단하고, PyMate에서 RAG 품질을 RAGAS로 진단해 본 경험이 GenON의 신규 Agent 생성·고객사 커스터마이징 단계에서 직접 활용될 수 있다고 봤습니다.

강점

응용통계학 전공으로 익힌 **가설 검증 절차를 AI 모델 의사결정에 옮기는 것**이 강점입니다. 'LLM 성능 부족' 가설을 RAGAS Context Precision으로 기각하고 임베딩 모델 한계로 방향을 튼 경험, LoRA v2 정확도 -3.2%p 하락을 데이터 노이즈로 해석해 19건만 재구성해 회복한 경험을 통해 **"증상이 아닌 단계별 지표를 먼저 본다"**는 절차를 정착시켰습니다.

입사 후 포부

단순히 모델을 붙이는 것이 아니라, 고객 도메인의 실패 패턴을 먼저 분류하고 **검증 레이어와 비용 구조까지 함께 설계**해 GenON 솔루션이 고객 환경에서 안정적으로 운영되는 데 기여하고 싶습니다. LLM Agent 신뢰성·RAG 품질·임베딩 학습을 한 솔루션 흐름으로 다루는 AI 엔지니어로 성장하겠습니다.

핵심역량

- **LLM Agent 신뢰성** — WorkFlow Agent(팀 4명) 검증·서빙 파이프라인 리드. GT QA **200건** 자체 벤치마크 + Guardrail·Confidence 보정으로 정확도 **37%→85%**, JSON 유효율 **70%→97%**, 과신 **0.92→0.78** 보정
- **RAG 품질 진단** — PyMate(팀 4명) RAG 평가·임베딩 담당. RAGAS로 LLM 교체 가설을 기각하고 임베딩 한계를 병목으로 식별 → **768D→3072D** 업그레이드 + 하이브리드 검색 도입으로 Context Precision **0.83→0.97**, Recall **0.70→0.79**
- **LoRA 학습·서빙** — Kanana-1.5-8B LoRA 데이터셋 재구성. v2 **-3.2%p** 하락을 데이터 노이즈로 진단해 오답 로그 **19건만** 재구성해 v3 회복. PagedAttention 기반 vLLM으로 RunPod A100에 서빙·운영
- **Python 백엔드·운영** — Seoul Culture Map(개인) Intent Routing으로 요청당 **\$0.020→\$0.003 (85%↓)**, 월 1만 요청 환산 **~\$200→~\$30**. 7개월간 3개 프로젝트에서 FastAPI/Django, REST **11종**, Docker, AWS EC2 운영

EDUCATION

가천대학교 응용통계학과

2020.03 ~ 2026.08 (졸업 예정)

SK Networks Family AI Camp (AI 부트캠프)

2025.09 ~ 2026.03 · AI/ML 엔지니어 집중 과정 수료 · 팀 4명 RAG-Agent 프로젝트 2건 (PyMate·WorkFlow Agent)

SKILLS

AGENT / LLM

LangGraph LangChain OpenAI API vLLM LoRA
PyTorch

RAG / EVAL

RAGAS Qdrant ChromaDB HyDE+BM25+RRF

SELECTED IMPACT

AGENT RELIABILITY
WORKFLOW AGENT

37% → 85%

Guardrail
+ Confidence 보정

GT QA 200건 LLM-as-a-judge 평가 기준. 환각 응답을 서비스 노출 전에 차단할 검증 게이트를 팀 리뷰로 확정·구현했습니다.

RAG QUALITY
PYMATE

0.83 → 0.97

RAGAS 기반 병목 진단

LLM 교체 가설을 RAGAS로 기각한 뒤 임베딩 모델 한계를 병목으로 확정했습니다.

AGENT ROUTING
SEOUL CULTURE MAP

\$0.020 → \$0.003

Intent Routing 비용 절감

단일 GPT-4o baseline 대비 요청당 85%↓, 월 1만 요청 환산 ~\$200 → ~\$30. 일상 대화·검색을 분기해 호출량 자체를 줄였습니다.

PROJECTS

WorkFlow Agent · 사내 업무 보조용 Agent 시스템

Team 4명 · 2026.02 ~ 04 · 8주

GitHub · 역할: AI 파이프라인(검증·서빙) 설계 책임 — 판단 Agent-Guardrail-LoRA 데이터셋-vLLM 서빙 (팀 4명)

팀 4명 중 검증 게이트 정책·LoRA 데이터셋·vLLM 서빙 구조의 아키텍처 설계를 맡았고, 구현 디테일은 코드 리뷰로 팀과 함께 확정했습니다.

고신뢰 응답(confidence **0.92**)을 서비스 노출 전에 차단하기 위한 Guardrail-Confidence 보정 게이트 설계 프로젝트입니다.

문제: 규정 판단 도메인에서 오답·환각·과신이 함께 발생해, 고신뢰 응답이 서비스 레이어를 통과할 위험이 있었습니다.

해결: 판단 Agent에 **다단계 Guardrail**과 **Confidence 보정**을 넣고, GPT/Claude/vLLM을 provider 패턴으로 교체 가능하게 통일했습니다.

결과: 규정 도메인 GT QA **200건** LLM-as-a-judge 평가 기준, LoRA 데이터셋 재구성을 거쳐 정확도 **37%→85%**, **JSON 유효율 70%→97%**를 확보하고 과신 confidence를 **0.92→0.78**로 보정했습니다.

배운 점: LoRA v2에서 **98건** 추가 학습 후 정확도 **-3.2%p** 하락을 데이터 노이즈와 모델 정확도의 상관관계로 분석해, v3에서는 오답 로그 **19건**만 재구성해 회복했습니다. Agent 신뢰성은 모델 성능보다 실패 처리 구조까지 함께 설계해야 안정화된다는 기준을 얻었습니다.

LangGraph vLLM LoRA Guardrail FastAPI Qdrant Docker RunPod A100

PyMate · RAG 학습 챗봇

Team 4명 · 2026.01 ~ 02 · 6주

GitHub · 역할: RAG 평가·임베딩 영역 담당 — 지표 정의·모델 업그레이드·Django 마이그레이션·EC2 운영 (팀 4명)

팀 4명 중 RAGAS 지표 정의·임베딩 모델 선정·프로덕션 마이그레이션을 전담했습니다.

"답변이 부정확하다"는 증상을 LLM 탓으로 돌리지 않고, RAGAS로 검색·생성을 분리 측정해 **임베딩 모델 한계**로 재정의한 프로젝트입니다.

문제: 검색과 생성 단계가 섞여 있어, 더 비싼 LLM으로 바뀌도 실제 원인을 놓칠 위험이 있었습니다.

해결: RAGAS로 **Context Precision 0.83** 병목을 식별한 뒤, 인프라 비용·성능 트레이드오프를 고려해 **768D→3072D** 임베딩 모델 업그레이드를 단행했습니다. HyDE+하이브리드 검색(BM25+RRF) 다단계 파이프라인을 설계하고, Reranker는 검색 품질을 높이되 레이턴시 병목을 최소화하도록 마지막 단계에만 적용했습니다.

결과: **Precision 0.83→0.97**, **Recall 0.70→0.79**로 RAG 검색 품질을 확보하고, Django 마이그레이션 후 AWS EC2 배포까지 연결해 운영 환경에 올렸습니다.

배운 점: "원인을 LLM 탓으로 돌리기 전 검색·생성을 분리 측정한다"는 절차를 RAG뿐 아니라 LoRA 학습 디버깅까지 같은 방식으로 적용하게 됐습니다 — 증상이 아닌 단계별 지표를 먼저 보는 습관을 얻었습니다.

RAGAS Qdrant Django Nginx AWS EC2

[GitHub](#) · 개인 · 문제 정의~배포까지 1인 진행

서울시 문화시설 **2,500+**개 데이터를 일회성 보고서로 끝내지 않고 사용자가 탐색·추천받을 수 있는 챗봇으로 확장하면서, 모든 요청을 단일 LLM에 태우는 비용 구조를 **Intent 라우팅**으로 분기한 PoC입니다.

문제: 분석 결과가 문서로만 남아 재사용되지 못했고, 단순 인사와 실제 검색 질의가 같은 LLM 파이프라인을 타며 비용이 커졌습니다.

해결: Intent→Retrieve→Generate 3-node로 분리해 일상 대화는 검색을 스킵하고, 검색형 요청만 SQL+ChromaDB RAG로 보내도록 설계했습니다.

결과: Intent Routing이 일상 대화의 불필요한 LLM 호출을 차단해 GPT-4o 단일 호출 baseline 대비 요청당 **\$0.020→\$0.003 (85%↓)**, 월 1만 요청 환산 **~\$200→~\$30** 절감. 로컬 임베딩+SSE 스트리밍으로 첫 토큰 **~500ms**, 응답 완료 평균 **~2.4s**, 세션 문맥 **10턴**을 확보했습니다.

배운 점: 효율적인 Agent 설계는 무조건적인 고성능 모델 호출보다 의도 파악(Intent Routing)을 통한 최적 경로 설정이 비용·성능 트레이드오프의 핵심임을 체득했습니다. 고객 환경별 트래픽 분포가 달라질 경우 분기 룰을 데이터 기반으로 재조정해야 한다는 운영 관점도 함께 얻었습니다.

[OpenAI API](#) [LangGraph](#) [ChromaDB](#) [FastAPI](#) [SSE](#) [SQLite](#) [React](#)

STATISTICS FOUNDATION

통계적 검증을 AI 모델 평가에 적용

응용통계학 지식을 AI 모델 결과값의 통계적 유의성 검증에 옮겨, 7개월간 3개 프로젝트에서 정확도·Precision/Recall·JSON validity·confidence·비용·latency **6종 지표**를 정의하고 RAGAS·LLM-as-a-judge로 측정 형태를 만든 뒤 구조 변경에 들어가는 절차를 유지합니다.

절차의 일관성 — 의사결정 변동성 줄이기

지표 정의 → 측정 → 구조 변경의 같은 순서를 7개월 3개 프로젝트에 일관 적용했습니다. **검색 병목·데이터 노이즈·호출 비용** 의사결정 모두 동일한 절차로 우선순위를 정해, 매 프로젝트마다 판단 기준이 흔들리지 않도록 했습니다.